

≡ Plan ▶

- Logique synchrone
- Bascule D
- SystemVerilog po...
- Mise en pratique (1)
- Automates (Machi...
- Mise en pratique (2)

ESAC - Numérique

Logique séquentielle

S. Chaudhuri, G. Duc, T. Graba, C.
Jabbour, U. Kühne, L. Sauvage

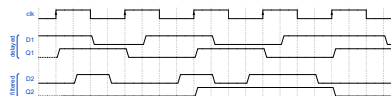
2023-2025

Cette partie du cours est consacrée à la
logique séquentielle synchrone. Nous
allons aborder deux aspects:

1. la construction de l'élément de
mémorisation sur *front*, la **bascule D**,
2. la description du comportement de la
logique séquentielle synchrone en
SystemVerilog

Logique synchrone

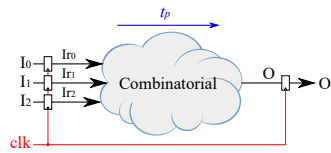
Rappel



Échantillonnage et mémorisation

L'élément de base de la logique
séquentielle synchrone permet
d'échantillonner une donnée à un instant
précis (un front d'horloge) et de mémoriser
sa valeur jusqu'au front suivant en
ignorant (filtrant) tous les changements
d'état intermédiaires.

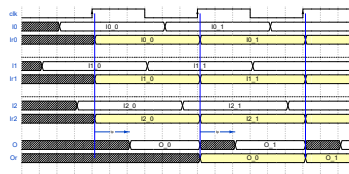
Réutiliser un bloc de logique combinatoire



Propagation dans la logique combinatoire

La logique séquentielle synchrone permet de garantir le fonctionnement correct de la logique combinatoire, on peut:

- présenter des données en entrée d'un bloc combinatoire,
- attendre que les calculs se propagent dans les blocs de logique combinatoire,
- simultanément, capturer le résultat et présenter de nouvelles entrées.



Propagation dans la logique combinatoire

Ce fonctionnement est garanti, tant que la période d'horloge est compatible avec le temps de propagation dans la logique combinatoire.

Dans la suite, nous allons voir les *principes* permettant de concevoir un tel élément mémorisant, actif sur fronts: **la bascule D** (ou le registre, s'il y en a

plusieurs) à partir des autres portes logiques.

Bascule D

L'élément mémorisant sur front



D flip-flop

clk	D	Q
↑	0	0
↑	1	1
↓	x	Q^{-1}
0	x	Q^{-1}
1	x	Q^{-1}

Ce qu'il faut retenir ici c'est que nous avons un élément mémorisant actif sur front. À chaque front montant (passage de 0 à 1) du signal d'horloge, on capture la valeur de l'entrée que l'on mémorise jusqu'au prochain front.

C'est l'élément de base de la logique synchrone que nous utiliserons dans la suite pour construire des applications. C'est le même type d'éléments, regroupé en registres, qui sont utilisés pour construire des éléments de calcul synchrones et ceci jusqu'aux processeurs.

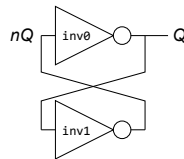
Dans la suite de ce chapitre, nous allons montrer comment pourrait être construite une bascule D avec les portes logiques précédemment vues.

Comment construire une

bascule D

Notez que ceci n'est donné qu'à titre indicatif. Les bascules D sont conçues au niveau transistor pour garantir leur comportement. Ce qui est montré dans ce chapitre n'est là que pour présenter les concepts.

Conserver et régénérer l'état électrique.



Point mémoire CMOS

Deux inverseurs tête-bêche permettent de conserver et régénérer le signal électrique Q . Tant ce système est alimenté, le signal nQ sera l'inverse de Q grâce à l'inverseur $inv1$, et l'inverseur $inv0$ nous régénérera un signal correct sur la sortie Q .

La fonction de transfert de la boucle composée des deux inverseurs est la fonction identité. Nous aurions pu utiliser un amplificateur non inverseur (un *buffer*), mais comme vu précédemment, en technologie CMOS, la porte *inverseur* est la porte la plus simple à construire (et un *buffer* serait composé de deux inverseurs, ce qui revient au même).

Cette structure est appelée un point mémoire statique (*statique* veut dire que tant que l'alimentation est présente, l'état est conservé). Cette structure réalise bien la fonction de mémorisation, mais telle quelle, il manque de quoi modifier la valeur

mémorisée (au-delà de ce qui sera présenté dans ce cours).

Aussi, il nous manque la possibilité de contrôler l’instant d’activation de cette mémoire (le front de l’horloge).

D-Latch

Considérons la structure suivante:



Boucle
avec un
MUX

ena	D	Q	
1	0	0	1
1	1	1	1
0	x	Q ₋₁	1

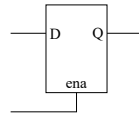
Avec un multiplexeur (MUX) rebouclé nous arrivons à construire un élément mémorisant. Quand la boucle est fermée, la fonction de transfert est aussi l’identité, ce qui permet de régénérer la valeur présente en sortie (un peu comme avec le point mémoire précédent).

Cette structure porte le nom de verrou (*Latch* en anglais, ou élément mémorisant actif sur niveau).

Maintenant, avec cette entrée de contrôle, nous avons deux états possibles:

- quand *ena* est à l’état haut (1), la valeur de l’entrée est présentée sur la sortie

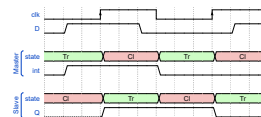
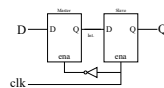
- on dit que le *Latch* est transparent
- quand *ena* est à l'état bas (0), la valeur de la sortie est maintenue
 - on dit que le *Latch* est bloqué (ou mémorisant)

Symbole du *Latch*

Nous avons maintenant un élément mémorisant que l'on peut contrôler, mais on ne maîtrise pas précisément l'instant où la capture de la valeur de l'entrée est faite. En effet, tant que le *Latch* est transparent, les changements sur l'entrée sont reproduits sur la sortie.

Dans l'étape suivante, nous allons voir comment utiliser ce composant pour obtenir le comportement désiré pour notre bascule D.

Structure Master/Slave



On chaîne de verrous (*Latches*) en reliant leur signal d'activation à un signal d'horloge périodique (*clk* dans le schéma) comme suit:

- le premier (appelé maître ou *master*) est commandé par l'inverse du signal d'horloge,
- le second (appelé esclave ou *slave*) est commandé par le signal d'horloge.

Quand le premier *Latch* est passant, le second est mémorisant et visse versa.

Dans le chronogramme, en vert les périodes où le *Latch* est passant et en rouge celle durant lesquelles il est bloqué.

1. Quand *clk* est à l'état bas: Le premier *Latch* est transparent et la valeur de l'entrée *D* est copiée sur le signal interne (*Int.* dans le schéma), mais elle n'apparaît pas sur la sortie *Q* qui garde son état précédent, car le second *Latch* est dans l'état mémorisant.
2. Quand *clk* est à l'état haut: Le second *Latch* devient transparent et la valeur du signal interne est copiée sur la sortie. Par contre, comme le premier *Latch* est bloqué, les changements de valeur du signal d'entrée ne sont pas propagés vers la sortie.

L'ensemble se comporte donc comme si on avait capturé la valeur du signal d'entrée au moment où le signal *clk* passe de 0 à 1 (au front).

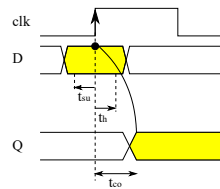
On a construit un élément mémorisant sur front!

Notez que les *vraies* Latches ne sont pas

construits avec des multiplexeurs, le concept de la structure maître/esclave est réaliste. Aussi, l'inversion du signal d'horloge est un peu plus complexe à mettre en place, du fait du retard qui serait ajouté sur l'un des chemins par un seul inverseur.

Finalement, vu l'importance des bascules D, il existe des structures optimisées au niveau transistors dont le comportement est validé par les concepteurs. On ne peut pas garantir ce comportement en assemblant des *Latches* sans descendre au niveau électrique!

Validité



Contraintes temporelles

Pour qu'une bascule échantillonne correctement la valeur de son entrée, il **faut** que celle-ci soit stable autour du front d'horloge. Ceci permet de capturer sa valeur puis de la propager vers la sortie de la bascule.

Les concepteurs des bascules D vont définir un intervalle temporel de stabilité, en dehors duquel le comportement de la bascule n'est plus garanti.

Autour du front, nous devons respecter:

- temps de pré-positionnement t_{su}

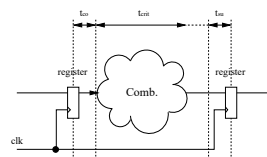
(*setup time*): le temps **avant** le front d'horloge durant lequel l'entrée n'a pas le droit de changer,

- temps de maintien t_h (*hold time*): le temps **après** le front d'horloge durant lequel l'entrée n'a pas le droit de changer.

De plus, t_{co} (*clock-to-output*) indiquera le délai entre le front de l'horloge et le moment où la donnée est valide à la sortie de la bascule. On peut voir ce temps comme le temps de propagation de la bascule.

Par construction, t_{co} est plus grand que le temps de maintien t_h , ce qui permet de directement connecter la sortie de la bascule sur l'entrée d'une autre bascule (pour construire des registres à décalage).

Chemin critique/fréquence



Relation temps de propagation/
période de l'horloge

$$T_{clk} \geq t_{co} + t_{crit} + t_{su} \text{ ou } F_{clk} \leq \frac{1}{t_{co} + t_{crit} + t_{su}}$$

On retrouve ici le schéma qui met en relation la période de l'horloge et le temps de propagation dans la logique combinatoire.

Avec t_{crit} le temps de propagation le plus long dans le bloc de logique combinatoire

(le pire cas).

Ces différents temps peuvent être retrouvés à partir des données associées aux portes logiques utilisées et des modèles plus ou moins précis (allant du modèle simple additif à la simulation électrique).

SystemVerilog pour la logique séquentielle

Description d'un registre/bascule-D

- *D-FF*
- *4-bit register*

```

module dff(
    input
    logic
    clk,

    input
    logic
    d,

    output
    logic q
);
    always_ff
    @(posedge
    clk)
    begin
        q <= d;
    end
endmodule

module reg4 (
    input
    logic
    clk,

    input
    logic
    [3:0]
    d,

    output
    logic
    [3:0] q
);
    always_ff
    @(posedge
    clk)
    begin
        q <= d;
    end
endmodule

```

La logique séquentielle synchrone est décrite dans des processus en utilisant le

mot-clé `always_ff` . Comme l'action décrite dans le processus est *exécutée* à chaque front montant de l'horloge on doit le préciser en ajoutant `@(posedge clk)` .

Note `@(...)` est appelé liste de sensibilité et permet de préciser l'évènement pour lequel le comportement décrit dans le processus doit être exécuté.

Notez aussi que l'opérateur d'affectation est différent (`<=`). Il est appelé opérateur d'affectation différée et il est **obligatoire** quand on décrit de la logique séquentielle synchrone. Il permet de décrire des changements d'états qui ne sont effectifs qu'après le front montant de l'horloge.

Affectation différée (`<=`)

```

module shift_reg
( input
  logic
  clk,

  input
  logic
  i,

  output
  logic q
);

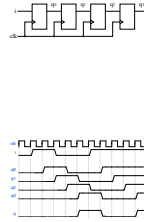
  logic q0, q1,
        q2, q3;

  always_ff
    @(posedge
     clk)
  begin
    q0 <= i;
    q1 <= q0;
    q2 <= q1;
    q3 <= q2;
  end

  assign q = q3;

endmodule

```



Ceci est un exemple de registre à décalage de 4 bits (4 bascules D). Notez que du fait de l'utilisation de l'opérateur d'affectation différée, les signaux ne changent qu'après le front d'horloge. Conceptuellement, les quatre affectations décrivent des actions qui sont **faites en parallèle**.

Donc ce bloc d'affectations est équivalent à:

```

begin
  q3 <= q2;
  q2 <= q1;

```

```

    q1 <= q0;
    q0 <= i;
end

```

Pour décrire de façon compacte le registre à décalage, nous aurions aussi pu exploiter l'opérateur de concaténation comme le montre l'exemple suivant.

```

module shift_reg ( input logic clk,
                  input logic i,
                  output logic q );

    logic [3:0] q_r;

    always_ff @(posedge clk)
    begin
        q_r <= {q_r[2:0],i};
    end

    assign q = q_r[3];

endmodule

```

Comme la sortie `q` est connectée au dernier registre, nous pouvons aussi écrire:

```

module shift_reg ( input logic clk,
                  input logic i,
                  output logic q );

    // La 4e bascule est q
    logic [2:0] q_r;

    always_ff @(posedge clk)
    begin
        {q,q_r} <= {q_r[2:0],i};
    end

endmodule

```

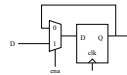
Le nombre de bascules correspond au

nombre de signaux modifiés par les affectations différées.

Travail à faire tester ces différentes versions sur [DigitalJS](https://esac.telecom-paris.fr/ELEC101-num/seance2.html).

Combiner logique combinatoire et logique séquentielle

- Une bascule D avec entrée d'activation (*enable*)



Bascule D avec enable

```
module dffe(
    input
    logic
    clk,

    input
    logic
    ena,

    input
    logic
    d,

    output
    logic q
) ;

always_ff
    @(posedge
    clk)

begin
    if(ena)
        q <= d;
end

endmodule
```

Ce code décrit le comportement d'une bascule D avec une entrée d'activation. La

sortie n'est mise à jour au front d'horloge
que si l'entrée d'activation est à l'état haut.

Dans le schéma, nous l'avons représenté
par une bascule précédée un multiplexeur.
Ce dernier, si l'entrée d'activation est à
l'état bas, redirige vers l'entrée de la
bascule sa sortie, ce qui fait qu'elle
n'évolue pas. Sinon, il dirige l'entrée vers
la bascule pour capturer une nouvelle
donnée.

Ce code est donc équivalent au code
suivant:

```
module dffe( input logic clk,
             input logic ena,
             input logic d,
             output logic q);

    logic q_next;

    assign q_next = ena? d : q;

    always_ff @(posedge clk)
    begin
        q <= q_next;
    end

endmodule
```

ou en utilisant le processus `always_comb`
pour la logique combinatoire:

```
module dffe( input logic clk,
             input logic ena,
             input logic d,
             output logic q);

    logic q_next;

    always_comb
    begin
```

```
    if(ena)
        q_next = d;
    else
        q_next = q;
end

always_ff @(posedge clk)
begin
    q <= q_next;
end

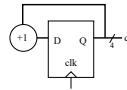
endmodule
```

Notez dans cette dernière version, que le bloc `if...else` doit être complet. En effet, pour de la logique combinatoire, tous les cas doivent être explicites.

Alors que pour de la logique séquentielle, les cas manquants veulent seulement dire que le signal ne change pas, il garde sa valeur, on mémorise. Ce qui veut dire que pour la logique séquentielle les cas implicites sont autorisés.

```
always_ff @(posedge clk)
begin
    if(ena)
        q <= d;
    // else
    //     q <= q; // implicite
end
```

Autre exemple: un compteur



Un
compteur
modulo
16

- Un compteur modulo 16

```
module cpt (
    input
    logic
    clk,

    output
    logic
    [3:0]
    q);

    always_ff
    @(posedge
    clk)

    begin
        q <= q + 1;
    end

endmodule
```

Le comportement de ce module est assez simple. Quelle que soit la valeur du signal q , au front d'horloge sa valeur est incrémentée. Aussi, comme le signal est sur 4 bits, s'il atteint la valeur 15, il passera automatiquement à 0 au cycle suivant. En suite, il reproduira, en permanence, la séquence des valeurs allant de 0 à 15. D'où le nom, *compteur modulo 16*.

Nous pouvons forcer le passage à 0 avant d'atteindre la valeur 15, pour avoir une séquence plus courte (modulo une valeur différente).

Par exemple, le code suivant représente un compteur modulo 10.

```
module cpt ( input logic clk,
              output logic [3:0]
              q);
```

```
always_ff @(posedge clk)
begin
    q <= q + 1;
    if(q == 9) q <= 0;
end

endmodule
```

Du fait de l'utilisation des affectations différées, le test `q == 9` se fait sur la valeur actuelle du registre `q`. Et nous aurons bien la séquence des valeurs allant de 0 à 9.

En fait ce code est équivalent à:

```
module cpt ( input logic clk,
              output logic [3:0] q);

always_ff @(posedge clk)
begin
    if(q == 9)
        q <= 0;
    else
        q <= q + 1;
end

endmodule
```

Problème avec cette construction, nous avons la garantie de parcourir toutes les valeurs prévues, par contre, au démarrage (à la mise sous tension) du système, on ne peut pas prédire l'état initial des registres.

Nous sommes obligés de prévoir un mécanisme pour forcer l'état des registres. Ce mécanisme d'initialisation, est appelé *Reset* (indépendamment de la valeur initiale souhaitée). Dans la section suivante nous expliquons comment le

décrire en SystemVerilog.

Initialisation/reset

- Remis à zéro synchrone



Remi
se à
zéro
synch
rone

- Un registre 4 bits
avec remise à zéro
synchrone.

```
module reg4( input
             logic clk,
             input
             logic rst,
             input
             logic [3:0]
             d,
             output
             logic [3:0]
             q);

    always_ff @(posedge
                clk)
    if(rst)
    begin
        q <= 0;
    end
    else
    begin
        q <= d;
    end
endmodule
```

Si le signal `rst` est à l'état haut, la sortie des portes ET est forcée à 0. Au front d'horloge suivant, la sortie du registre passera forcément à 0. Cet état sera conservé tant que l'entrée `rst` reste à 1.

Ce mécanisme de remise à 0 est appelé reset synchrone, car son effet n'est visible qu'après un front d'horloge. Il est simple à mettre en œuvre puisque il ne nécessite

que des portes logiques et bascules D standards.

Notez que dans le code nous avons séparé le comportement en deux parties, l'une correspondant à la remise à 0 et l'autre correspondant au fonctionnement normal. Il est important de conserver ce style d'écriture pour permettre aux outils d'identifier la partie correspondant au reset.

Remise à zéro asynchrone

- sur niveau haut
- sur niveau bas

```

module dffa_1(
    input
    logic
    clk,

    input
    logic
    rst,

    input
    logic
    d,

    output
    logic
    q);

    always_ff
        @(posedge
        clk or
        posedge
        rst)

    if(rst)
    begin
        q <= 0;
    end
    else
    begin
        q <= d;
    end
endmodule

module dffa_0(
    input
    logic
    clk,

    input
    logic
    nrst,

    input
    logic
    d,

    output
    logic
    q);

    always_ff
        @(posedge
        clk or
        negedge
        nrst)

    if(~nrst)
    begin
        q <= 0;
    end
    else
    begin
        q <= d;
    end
endmodule

```

Un signal de remise à zéro asynchrone agit immédiatement sur la sortie du registre. Ce comportement est décrit en SystemVerilog et ajoutant le signal de reset à la liste de sensibilité du processus `always_ff`. Plus précisément, il faut ajouter l'évènement qui rend le reset actif:

- son passage à 1 s'il est actif à l'état haut,
- son passage à 0 s'il est actif à l'état bas.

La syntaxe de SystemVerilog n'a pas prévu de mot-clé dédié pour le reset et a réutilisé le mot-clé prévu pour l'horloge.

Malgré cela, il ne faut pas l'interpréter comme une seconde horloge, les bascules n'ont qu'une seule horloge!

Aussi, la séparation du comportement en deux parties distincte est **obligatoire**.

La mise en œuvre de ce type de reset électroniquement se fait forcément au niveau transistor dans la bascule D qui sera différente d'une bascule D standard. Elle est souvent plus compacte que l'ajout d'une porte logique supplémentaire. On le représente juste par une entrée supplémentaire de la bascule.

Mise en pratique (1)

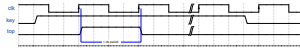
Compteurs

- Écrire et tester sur DigitalJS un compteur modulo 13 avec:
 - une remise à zéro synchrone active sur niveau bas
 - une remise à zéro asynchrone active sur niveau bas
- Ajoutez la possibilité de stopper et de reprendre le comptage (*enable*).
- Ajoutez la possibilité charger le compteur avec une valeur provenant d'une entrée supplémentaire.

Détecteur de front

Nous voulons détecter le passage de 0 à 1
d'un signal `key` venant du monde
extérieur dans un système synchrone à
l'horloge `clk`.

`top` le signal en sortie indique ce
changement d'état en passant à 1 durant
exactement 1 cycle d'horloge à chaque
fois que l'entrée passe de 0 à 1.



1. Construisez le schéma, puis testez le code SystemVerilog.
2. Modifiez le résultat pour détecter le **passage de 0 à 1**, puis pour détecter les **deux changements**.

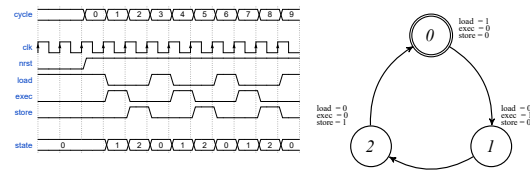
Automates (Machines à états)

Génération d'une séquence de signaux à partir d'un diagramme d'états d'un système synchrone.

Automate cyclique

Si le comportement recherché est périodique et cyclique, on peut utiliser un compteur pour représenter les états du système.

Exemple



Exemple (suite)

```

module mae ( input
    logic
    clk,
    nrst,
    output
    logic
    load,
    exec,
    store );

    logic [1:0] state;

    always_ff
        @(posedge
        clk or
        negedge
        nrst)

    if(!nrst)
        state <= 0;
    else
        if(state == 2)
            state <=
                0;
        else state <=
            state +
            1;

    assign load =
        (state ==
        0);
    assign exec =
        (state ==
        1);
    assign store =
        (state ==
        2);

endmodule

```

Un compteur qui
représente l'état du
système:

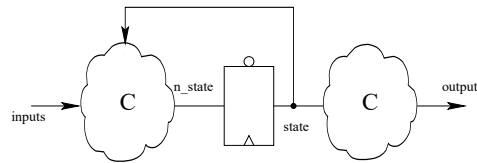
- Au reset on commence à 0
- S'incrémente à chaque cycle sinon
- Modulo le nombre d'états dans le cycle

Les sorties sont
calculées en
fonction de l'état:

- combinatoire en fonction de la valeur du compteur

Généralisation

Nous pouvons généraliser ce concept à des automates non-cycliques pour lesquels les transitions d'états ne sont pas régulières et dépendent d'entrées du système.



Nous avons besoin de construire un système séquentiel très générique avec les éléments suivants:

- un registre pour stocker l'état

Ce registre évoluera à chaque front d'horloge et conservera un état entre 2 fronts.

- un bloc combinatoire qui calcule l'état futur en fonction de l'état et des entrées

Ce bloc remplace l'additionneur de l'automate cyclique. Sa construction va dépendre des changements d'états que nous voulons permettre.

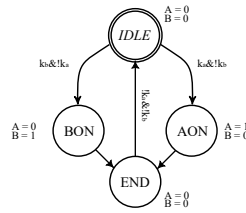
Nous verrons dans l'exemple suivant comment exprimer ces changements d'états et comment les traduire simplement en SystemVerilog.

- un bloc combinatoire qui calcule les sorties en fonction de l'état

Ce montage est nommé machine à états

de Moore.

Exemple



Un système avec deux entrées k_a et k_b et deux sorties A et B .

Si k_a ou k_b passe à 1, A ou B passe à 1 durant un cycle, puis on attend que k_a et k_b repassent à zéro.

Ce type de diagramme est appelé diagramme d'états. C'est un diagramme orienté dans lequel, nous avons:

- des cercles représentant les états du système,
- des flèches reliant ces cercles représentant les changements d'états.
 - on appellera cela les transitions d'états

Ces transitions peuvent être:

- conditionnées par l'état des entrées
 - on indiquera la condition sur la flèche
 - si la condition est fausse on reste dans l'état de départ
 - si non on pourra changer d'état
- ou inconditionnelles
 - sans condition sur la flèche
 - on changera forcément d'état

Il faudra veiller à ce qu'il n'y ait pas de

contradiction entre les différentes transitions partant d'un état et qu'il n'y a qu'une seule transition qui peut être prise pour toutes les valeurs des entrées.

Notez que comme l'état est sauvegardé dans un registre, le changement d'état ne peut avoir lieu qu'au front d'horloge suivant.

Exemple (code SystemVerilog)

```

module fsm(input      ....
            clk,
            nrst,      always_ff@(posedge
                        clk or
                        negedge
                        nrst)
            output      if(!nrst)
            A, B);      state <=
                        IDLE;

// On peut
// nommer
// les
// états
// en utilisant
// un type
// énuméré
enum logic[1:0]
{IDLE,AON,BON,END}
state;

// Les sorties
// en
// fonction
// de
// l'état
assign A =
(state
==
AON);
assign B =
(state
==
BON);

// Le registre
// d'état et
// les
// transitions endmodule

// Avec:
// * L'état
//   initial
// * Les
//   transitions
//   à partir
//   des états

...

```

Pour décrire de façon lisible les états et les transitions, nous utilisons une nouvelle construction de SystemVerilog, les types

énumérés déclarés en utilisant le mot clé

`enum` .

Dans l'exemple, nous déclarons le signal qui va contenir l'état comme:

```
enum logic[1:0] {IDLE, AON, BON, END}  
    state;
```

- `enum` permet d'utiliser des noms à la place de valeurs numériques,
 - ces noms ne servent qu'à rendre le code lisible, à la fin le registre contiendra des valeurs *binaires*,
- `logic[1:0]` indique que nous avons besoin de 2 bits pour encoder les 4 valeurs possibles,
- `state` le nom du signal du registre
- `{IDLE, AON, BON, END}` sont les noms que nous utiliserons dans le code,
 - ces noms correspondront aux valeurs 0,1,2 et 3,
 - ces noms ne peuvent être utilisés qu'avec le signal `state` .

Dans la suite de l'exemple, nous avons un processus synchrone pour le registre d'états et les transitions. Au reset, nous forçons l'état initial. En suite, à chaque front d'horloge, nous évaluons les transitions. Ces transitions représentent le calcul combinatoire de l'état futur et le remplacement de l'état par ce dernier à chaque front d'horloge.

Nous pouvons aussi décomposer cette partie et séparer le registre du bloc combinatoire en faisant apparaître

explicitement le signal représentant l'état futur. Le code deviendrait alors:

```
module fsm( input clk, nrst,
            input Ka, Kb,
            output A, B);

    // Les signaux pour le registre et
    // l'état futur
    // doivent être déclarés en même
    // temps
    enum logic[1:0] {IDLE, AON, BON, END}
        state, n_state;

    // Les sorties en fonction de
    // l'état
    assign A = (state == AON);
    assign B = (state == BON);

    // Le registre d'état
    // * L'état initial au reset
    // sinon, on prend la sortie du
    // bloc combinatoire
    always_ff@(posedge clk or negedge
               nrst)
        if(!nrst)
            state <= IDLE;
        else
            state <= n_state;

    // Le calcul de l'état futur
    always_comb
    begin
        // par défaut on reste dans
        // l'état courant
        // dans la suite nous
        // n'exprimons que ce qui nous
        // ferait changer d'état
        n_state = state;
        case(state)
            IDLE: begin
                if(Ka & !Kb) n_state =
                    AON;
                if(!Ka & Kb) n_state =
                    BON;
            end
            AON: n_state = END;
            BON: n_state = END;
        endcase
    end
endmodule
```

```
END: if (!Ka && !Kb) n_state  
    = IDLE;  
endcase  
end  
  
endmodule
```

Mise en pratique (2)

Détecteur de front

Reprendre l'exercice en utilisant le formalisme de machines à états.

Filtrage à moyenne glissante (en préparation de la prochaine séance)

Nous recevons une séquence de données codées sur 8 bits.

1. Réalisez un filtre calculant la somme des 4 derniers échantillons reçus.
2. Réalisez un filtre calculant la moyenne des 4 derniers échantillons reçus.
3. Optimisez la structure pour limiter le nombre d'additionneurs nécessaires à moins de 3.

Pour tester votre solution en SystemVerilog sur [DigitalJS](#) vous disposez d'un environnement de test:

- Dans les exemples, choisissez **Moving Average**
 - vous disposez ainsi d'un

environnement de test avec un
générateur d'échantillons

- Implémentez vos solutions dans le module `filter`.

Conversion série parallèle

- Nous recevons, de manière synchrone, une séquence de données `data_in` codées sur 1 bit.
- À chaque cycle d'horloge un signal `en_in` indique si le bit `data_in` est une donnée valide.
- Nous voulons transformer cette séquence en séquences de données `data_out` de 4 bits, accompagnée d'un signal `en_out` indiquant si la donnée `data_out` est valide.

1. Réalisez un chronogramme montrant un exemple de transmission.
2. Déterminez le ou les signaux supplémentaires nécessaires au fonctionnement du dispositif.
3. Déterminez les éléments logiques nécessaires à la réalisation du dispositif.
4. Faites un schéma.
5. Écrivez le code SystemVerilog équivalent.

Pour tester votre solution en SystemVerilog sur [DigitalJS](#) vous disposez d'un environnement de test:

1. Dans les exemples, choisissez **Serial**

to Parallel

- vous disposez ainsi d'un environnement de test avec un générateur d'échantillons

2. Implémentez vos solutions dans le module `serial2parallel` .

[Retour à la page principale](#)

© Copyright 2022-2025, Télécom Paris.



Le contenu de cette page est mis à disposition selon les termes de la licence [Creative Commons Attribution - Partage dans les Mêmes Conditions 4.0 International](#).